

Mini-Project 2: Choose Your Own Adventure Programming

Russell Helmstedter

09/07/2026

Contents

1	Propose a Student Programming Assignment	2
1.1	Build a GPA Calculator	2
1.2	Success Criteria	2
1.3	Optional Extensions	2
2	Write a Detailed Specification	3
2.1	Overview	3
2.2	Program Requirements	3
2.3	Example Input and Output	3
2.4	IPO Description	4
2.5	Variables and Data	4
2.6	Algorithm	4
2.7	Constraints	5
3	Create the Solution	5
3.1	Source Code	5
3.2	Sample Run	6
3.3	Testing Evidence	6
3.4	Extension Solutions	7
4	Teacher Notes	8
4.1	Estimated Work Time	8
4.2	Prerequisite Knowledge	8
4.3	Vocabulary	8
4.4	A Likely Misconception	8
4.5	Supporting Students Who Need Help	9
4.6	Extending for Students Ready for a Challenge	9
4.7	Accessibility	9
5	Reflection	9
5.1	Why is this appropriate for grades 9-12?	9
5.2	Which Chapter 5 and 6 concepts does it assess?	9
5.3	How does it demonstrate input, processing, and output?	10
5.4	What did I discover while building and testing it?	10
5.5	What would I change before using it?	11
6	Files	11

1 Propose a Student Programming Assignment

1.1 Build a GPA Calculator

Every quarter you find out your GPA, and every quarter somebody asks how it was calculated. It is not complicated. Each letter grade is worth a number of points, you add up the points from all of your classes, and you divide by how many classes you took. Your job is to write the program that does it: ask for the grade points earned in each of six classes, add them, divide, and report the GPA rounded to two decimal places.

The goal is to build a program that takes in several numbers, combines them with arithmetic, and reports a result you could not have worked out at a glance. Along the way you will find out why a number typed on a keyboard is not yet a number the computer can add.

1.2 Success Criteria

- ☐ It asks for grade points for all six classes and waits for each answer.
- ☐ Every answer is converted from text to a whole number with `int()`.
- ☐ `NUMBER_OF_CLASSES` is defined at the top as a constant, named in capital letters.
- ☐ The total is calculated by adding the six values, not typed in by hand.
- ☐ The GPA is calculated by dividing, and is rounded to two decimal places.
- ☐ Changing `NUMBER_OF_CLASSES` at the top changes the arithmetic wherever it is used.
- ☐ Comments mark the input, process, and output sections.
- ☐ Both output lines are built with f-strings.
- ☐ It runs top to bottom without errors after **Runtime -> Restart and run all**.

1.3 Optional Extensions

Pick one. Each uses something we have not covered yet.

1. **Let the user type letter grades.** Typing 4 when you mean an A is backwards. People have letter grades, not point values. Make the program accept A through F and work out the points itself. You will need somewhere to store what each letter is worth. Ask about *dictionaries*.
2. **Honor roll.** Add a constant `HONOR_ROLL_GPA` set to 3.5. Use an `if` statement so a student who made honor roll is congratulated, and a student who did not is told how many more grade points it would have taken.
3. **Any number of classes.** Use a `while` loop and a list so the user can enter as many grades as they want and press enter on a blank line to finish. Divide by how many were actually entered.

2 Write a Detailed Specification

2.1 Overview

Grade band	9-12
Platform	Python on Google Colaboratory
Assignment	GPA Calculator
Concepts	Variables, assignments, arithmetic expressions, type conversion, data types, constants

2.2 Program Requirements

1. Define a constant `NUMBER_OF_CLASSES` with the value 6.
2. Print a welcome line and two lines explaining the grading scale.
3. Prompt the user six times, once per class, and store each answer.
4. Convert each of the six stored answers from a string to an integer.
5. Calculate `total_points` as the sum of the six converted values.
6. Calculate `gpa` as `total_points / NUMBER_OF_CLASSES`.
7. Round `gpa` to two decimal places.
8. Print the total grade points and the number of classes.
9. Print the GPA.

2.3 Example Input and Output

A typical case. The student earned A, B, A, B, C, A.

```
Welcome to the GPA Calculator.
Enter the grade points you earned in each class.
A is 4, B is 3, C is 2, D is 1, F is 0.
Grade points for class 1: 4
Grade points for class 2: 3
Grade points for class 3: 4
Grade points for class 4: 3
Grade points for class 5: 2
Grade points for class 6: 4
```

```
You earned 20 grade points in 6 classes.
Your GPA is 3.33.
```

A boundary case. Straight A's, the highest result the program can produce.

```
Grade points for class 1: 4
Grade points for class 2: 4
```

```
Grade points for class 3: 4
Grade points for class 4: 4
Grade points for class 5: 4
Grade points for class 6: 4
```

```
You earned 24 grade points in 6 classes.
Your GPA is 4.0.
```

An error case, included because students will hit it in the first five minutes. Typing the letter grade instead of its point value stops the program.

```
Grade points for class 1: A
```

```
ValueError: invalid literal for int() with base 10: 'A'
```

The student's instinct is right and the program is the limited one. Extension 1 exists to fix it.

2.4 IPO Description

Inputs. Six strings entered by the user, one per class, each expected to be a digit from 0 to 4.

Processing. Six type conversions from string to integer. One addition producing the total. One division producing the GPA. One call to `round()`.

Outputs. Two printed lines, both built with f-strings. The first reports the total grade points and how many classes they came from; the second reports the GPA.

2.5 Variables and Data

Name	Stores	Type	Kind
<code>NUMBER_OF_CLASSES</code>	How many classes are on the schedule	int	Constant
<code>points_1 ... points_6</code>	One class's grade points	str then int	Variable
<code>total_points</code>	The six values added together	int	Variable
<code>gpa</code>	Total divided by the number of classes	float	Variable

`points_1` through `points_6` change type partway through the program, which is the clearest illustration of type conversion the assignment offers. `gpa` is a float even though nobody typed a decimal point, because dividing an int by an int in Python produces one.

2.6 Algorithm

```
BEGIN
```

```
    SET NUMBER_OF_CLASSES to 6
```

```
    DISPLAY welcome message
```

```
    DISPLAY the grading scale
```

```
    FOR each of the six classes (written out, not looped):
```

```
        PROMPT for grade points
```

```

STORE the answer

FOR each of the six answers (written out, not looped):
    CONVERT the answer to an integer

SET total_points to the sum of the six answers
SET gpa to total_points / NUMBER_OF_CLASSES
SET gpa to gpa rounded to 2 decimal places

DISPLAY total_points and NUMBER_OF_CLASSES
DISPLAY gpa
END

```

2.7 Constraints

Students must define `NUMBER_OF_CLASSES` as a constant at the top, convert every input with `int()` before using it in arithmetic, mark the input, process, and output sections with comments, and build both printed lines with f-strings.

Students do not need conditional statements, loops, or functions. None have been covered, and the required program is designed so that none are needed.

Students may not use lists, dictionaries, or imported libraries. Twelve near-identical lines is deliberately repetitive, and that repetition is the argument for the extensions.

3 Create the Solution

3.1 Source Code

```

"""GPA Calculator."""

# CONSTANTS
NUMBER_OF_CLASSES = 6

# INPUT
print("Welcome to the GPA Calculator.")
print("Enter the grade points you earned in each class.")
print("A is 4, B is 3, C is 2, D is 1, F is 0.")

points_1 = input("Grade points for class 1: ")
points_2 = input("Grade points for class 2: ")
points_3 = input("Grade points for class 3: ")
points_4 = input("Grade points for class 4: ")
points_5 = input("Grade points for class 5: ")
points_6 = input("Grade points for class 6: ")

# PROCESS
# input() always hands back a string, so every answer needs converting first.
points_1 = int(points_1)
points_2 = int(points_2)
points_3 = int(points_3)

```

```

points_4 = int(points_4)
points_5 = int(points_5)
points_6 = int(points_6)

total_points = points_1 + points_2 + points_3 + points_4 + points_5 + points_6

# Dividing an int by an int gives a float, which is what a GPA needs.
gpa = total_points / NUMBER_OF_CLASSES
gpa = round(gpa, 2)

# OUTPUT
print(f"You earned {total_points} grade points in {NUMBER_OF_CLASSES} classes.")
print(f"Your GPA is {gpa}.")

```

3.2 Sample Run

The program as it actually runs, with no blank line between the last prompt and the first result.

```

Welcome to the GPA Calculator.
Enter the grade points you earned in each class.
A is 4, B is 3, C is 2, D is 1, F is 0.
Grade points for class 1: 4
Grade points for class 2: 3
Grade points for class 3: 4
Grade points for class 4: 3
Grade points for class 5: 2
Grade points for class 6: 4
You earned 20 grade points in 6 classes.
Your GPA is 3.33.

```

3.3 Testing Evidence

Rather than retype output by hand, I wrote a harness that feeds each specification test case to the real program through standard input and compares the output against what the specification promises. The file is `test_gpa_calculator.py`. Its actual output follows.

```

[PASS] Typical case: A, B, A, B, C, A
      input: 4, 3, 4, 3, 2, 4
      You earned 20 grade points in 6 classes.
      Your GPA is 3.33.

[PASS] Every point value used: 3, 2, 4, 1, 0, 3
      input: 3, 2, 4, 1, 0, 3
      You earned 13 grade points in 6 classes.
      Your GPA is 2.17.

[PASS] Boundary case: straight A's (highest possible)
      input: 4, 4, 4, 4, 4, 4
      You earned 24 grade points in 6 classes.
      Your GPA is 4.0.

```

```
[PASS] Boundary case: straight F's (lowest possible)
input: 0, 0, 0, 0, 0, 0
You earned 0 grade points in 6 classes.
Your GPA is 0.0.
```

```
[PASS] Error case: a letter is not a number
input: A, 3, 4, 3, 2, 4
ValueError: invalid literal for int() with base 10: 'A'
```

5 of 5 test cases passed.

3.4 Extension Solutions

Not part of the required assignment. I wrote and ran all three so the teacher notes describe code that works.

Extension 1 replaces the input and processing sections with a lookup table.

```
LETTERS_TO_POINTS = {"A": 4, "B": 3, "C": 2, "D": 1, "F": 0}

grade_1 = input("Letter grade for class 1: ")
# ... five more

points_1 = LETTERS_TO_POINTS[grade_1]
# ... five more
```

Grades of A, B, A, B, C, A produce the same 20 points and the same 3.33, which is the point: the arithmetic did not change, only what the user is allowed to type. A lowercase **a** now fails with `KeyError: 'a'` rather than `ValueError`.

Extension 2 replaces the output section.

```
HONOR_ROLL_GPA = 3.5

print(f"Your GPA is {gpa}.")
if gpa >= HONOR_ROLL_GPA:
    print("Congratulations, you made honor roll.")
else:
    points_needed = HONOR_ROLL_GPA * NUMBER_OF_CLASSES - total_points
    print(f"You need {points_needed} more grade points to make honor roll.")
```

A GPA of exactly 3.5 congratulates, since the comparison uses `>=`. The typical case reports `You need 1.0 more grade points to make honor roll`.

Extension 3 replaces the input and processing sections with a loop.

```
points = []
print("Enter grade points one at a time. Press enter on a blank line to finish.")
)
while True:
```

```
    answer = input("Grade points: ")
    if answer == "":
        break
    points.append(int(answer))

total_points = sum(points)
gpa = round(total_points / len(points), 2)
```

Four grades of 4, 3, 4, 2 give You earned 13 grade points in 4 classes. and Your GPA is 3.25. Seven grades work equally well. `NUMBER_OF_CLASSES` is no longer needed at all, which is worth asking students about.

4 Teacher Notes

4.1 Estimated Work Time

One 55-minute period. Most of it is typing. The conceptual work is concentrated in about four minutes, which is why I would run this as a whole-class build rather than silent independent work.

4.2 Prerequisite Knowledge

Chapter 5 and Chapter 6 through section 6.14, plus one prior program using `input()`, `print()`, variables, and f-strings.

4.3 Vocabulary

Constant, variable, identifier, assignment, expression, data type, string, integer, float, type conversion, rounding.

4.4 A Likely Misconception

Students believe that because they typed a number, they have a number. They have a string that looks like one. This surfaces two ways.

The louder way: a student types `A` instead of `4` and gets `ValueError: invalid literal for int() with base 10: 'A'`. Python names the exact value it could not convert, which makes it one of the friendlier errors to read.

The quieter way: a student skips the conversions entirely. The six inputs are joined end to end rather than added, `total_points` becomes the string `"434324"`, and the program stops at the division with `TypeError: unsupported operand type(s) for /: 'str' and 'int'`. The message is precise, but only if you already know what `'str'` means, which is the whole lesson. I would put both tracebacks on the board before anyone hits them.

4.5 Supporting Students Who Need Help

Hand out a notebook with the constant and the six prompts already written, so the student supplies only the conversions and the arithmetic. This removes typing without removing thinking.

For students stuck on type conversion specifically, put "4" + "3" and 4 + 3 in adjacent cells and run both. Seeing 43 and 7 side by side does more than an explanation.

4.6 Extending for Students Ready for a Challenge

The three extensions are ordered by how badly students want them rather than by chapter. Extension 1 is first because it is the one they ask for unprompted, within minutes, as soon as the program rejects an A. It needs a dictionary, which the book has not reached, so the honest answer to a student who asks is that they have found the reason dictionaries exist. Extensions 2 and 3 preview Chapters 7 and 8. Past all three, the natural question is weighted GPA.

4.7 Accessibility

Colab runs in any browser, works with a screen reader, and inherits browser font sizing. The pre-populated notebook cuts typing for students who need that.

5 Reflection

5.1 Why is this appropriate for grades 9-12?

The arithmetic is not the challenge, and that is deliberate. Every student already knows what a GPA is and roughly how it is computed, so nothing in the problem domain competes for attention with the programming. What is genuinely new is that a value has a type, that the type determines what can be done with it, and that "4" and 4 are different things that look identical on a screen. That is abstract, and it is within reach for a high schooler in a way it would not be for a sixth grader.

The context also does work a temperature converter would not. Students care about their GPA. The first time the program prints a number they did not expect, the argument in the room is not about Python.

5.2 Which Chapter 5 and 6 concepts does it assess?

Alignment to the textbook is as follows:

Section	Where it appears
5.1 Programming (general)	The input-process-output structure of the whole program
5.3 Comments and whitespace	Required comment headers marking the three stages
5.5 Code and pseudocode	The algorithm in the specification
6.1 Variables and assignments	Eight assignment statements
6.2 Variables (integer)	The six point values and the total
6.3 Identifiers	Naming rules for variables and constants
6.4, 6.5 Arithmetic expressions	The sum and the division
6.7 Floating-point	<code>gpa</code>
6.8 Math functions	<code>round()</code>
6.11 Type conversions	Six <code>int()</code> calls
6.13 Data types	Values that arrive as <code>str</code> and become <code>int</code> then <code>float</code>
6.14 Constants	<code>NUMBER_OF_CLASSES</code>

5.3 How does it demonstrate input, processing, and output?

The three stages are separated in the source by comment headers, which section 5.3 asks for, and they are also separated in time when the program runs. Six prompts appear and wait. Then nothing visible happens for the length of nine statements. Then two lines appear at once.

That silent middle is the part worth naming. Students tend to believe a program is doing something only when the screen changes, and this program spends its most important moments doing nothing they can see. The six conversions in particular are invisible and load-bearing: remove them and the program does not produce a wrong answer, it stops running.

5.4 What did I discover while building and testing it?

First, the error case matters more than either boundary case. Straight A's and straight F's both work and tell me little. Typing A into a prompt that wants 4 stops the program cold, and it is what most students will actually experience.

Second, `round()` and a format specifier are not the same tool, and I had conflated them in an early draft. `round(gpa, 2)` changes the value stored in `gpa`; `f"{gpa:.2f}"` changes only what is displayed. The visible consequence is that straight A's prints 4.0 rather than 4.00. I kept `round()` because it matches how I would write this outside a classroom, and the inconsistent decimal width is a reasonable thing for a student to notice and ask about.

Third, and most usefully: I originally wrote this with letter grades and a dictionary, because that is how my own version works. Checking the assignment against the zyBook's section list showed that dictionaries appear nowhere in Chapters 5 or 6, and arrays do not arrive until Chapter 9. The lookup table was three chapters further out of scope than the `if` statement I had already been careful to exclude. Reversing it, so that students type numbers and the dictionary becomes the extension they ask for, produced a better assignment than either version I had before. The frustration of being told A is not a number is now the motivation for the next chapter rather than a detail I had smoothed over.

5.5 What would I change before using it?

The twelve near-identical lines are the weakest part. They are honest, since students have no loop yet, and the repetition is the argument for Extension 3. But twelve is enough to feel like busywork rather than a hint, and four classes would make the same point with a third less typing. I may cut it.

I would also like to know whether Extension 1 lands as an invitation or as a wall. Telling a student that the fix for their complaint is a topic we have not reached is either motivating or deflating, and I cannot predict which from my desk. That is what I would watch for the first time I run this.

6 Files

- `gpa_calculator.py` — the complete solution, tangled from this document
- `test_gpa_calculator.py` — the test harness that produced the testing evidence
- `GPA_Calculator_Solution.ipynb` — the Colab notebook version